

Case Mibo Smart

Documento de produto e arquitetura

Versão escrita da apresentação de 24/09/2026. O detalhe vive no repositório: `docs/adr/` (27 decisões, com as opções recusadas), `docs/specs/SPEC.md` (critérios de aceite), `docs/api-contract.md` (o contrato como observado), `AI-LOG.md` (uso de IA).

1. Objetivo

App Kotlin Multiplatform que consome a plataforma Open Casa Inteligente: token de acesso, lista de dispositivos com paginação e filtro, vídeo ao vivo de câmeras e controle de fechadura. Android é o alvo principal; iOS existe para provar que o compartilhamento é real, não rótulo.

2. Planejamento

Os dias

Dia	Foco
Sáb 19	Stack, bibliotecas, padrão de arquitetura
Dom 20	Sondar a API, ler avaliações das lojas, ver vídeos do app oficial em uso
Seg 21	Implementação (96 commits)
Ter 22	Implementação e identidade visual (26 commits)
Qua 23	Implementação e correções contra a API real (21 commits)

Quatro ondas

Onda	Escopo
0	<i>Walking skeleton</i> : CI, esqueleto de módulos, SPEC, ADRs, testes de arquitetura
1	Sessão e token
2	Lista, vídeo ao vivo e fechadura, em paralelo
3	Histórico, renovação de token, iOS, módulo Java, R8

A fatia é vertical

Uma fatia atravessa `domain → data → app → tela` e entrega algo observável no fim. Teto de 400 linhas executáveis de produção por PR, medido por script. Testes são contados e não entram no teto, porque quem os limita são os critérios de aceite. Uma issue por fatia, um PR por issue, merge por humano.

3. Tecnologia: o que não é escolha padrão

Ktor, Koin e SQLDelight são o previsível de um projeto KMP. O que vale explicar é o resto.

Escolha	Por quê
Konture	Arquitetura como teste que quebra o build, não como convenção escrita num README
Roborazzi	42 capturas de tela geradas a partir das telas que já existem
Kover	A meta de 80% de cobertura só existe se for medida
Módulo Java puro	Interoperabilidade provada por um teste escrito em Java, não afirmada
Media3 / WKWebView	Código de plataforma só na superfície do player

Konture e Kover vieram das duas preocupações que o time citou na entrevista: manter a arquitetura ao longo do tempo, e cobertura de testes alta.

Regra que governa a lista: nenhuma dependência entra para “fazer compilar”. Ela entra com uma linha de ADR dizendo o que resolve e o que foi recusado no lugar.

4. Arquitetura

Dependências só para dentro

Módulo	O que é
domain	Kotlin puro: modelos, contratos, erros tipados. Sem Ktor, Koin, SQLDelight, Android ou Compose
data	O parceiro mora aqui, e só aqui: DTOs, mapeadores, cache, cofre do token
app	Casos de uso, ViewModels e telas. Gera o framework do iOS

Regras do Konture

Cinco regras rodam como teste JVM em todo PR, antes de qualquer teste de negócio:

Regra	O que proíbe
1	O domínio depender de framework ou de persistência
2	Dependência apontando para fora, e ciclo entre módulos
4	<code>android.*</code> ou <code>java.*</code> no código compartilhado
5	Repositório declarado no domínio que não seja interface
8	Código de plataforma fora de um pacote <code>platform</code>

Por que módulo por camada, e não por feature

Duas formas foram comparadas antes de escrever código:

- **B, por camada** (escolhida): três módulos, domínio, dados e app. As quatro features são pastas dentro do app, já isoladas: nenhuma importa a outra.
- **C, por feature**: oito a doze módulos, um por feature mais os de base. Cada um vira um alvo compilado para o iOS.

Um desenvolvedor e quatro telas: o custo de C é real e o ganho dele, times trabalhando em paralelo, não existe aqui. E promover para C mais tarde é mover pastas, porque as fronteiras entre features já estão respeitadas desde o começo.

5. Organização do GitHub

- **Issues** criadas por fatia, com critérios de aceite e os arquivos que a fatia toca.
- **Pull requests** mapeados um a um para as issues, com o porquê no corpo.
- **Épicos e milestones** por onda, organizados em um Project.
- **CI em três estágios**: arquitetura e testes, APK e lint, iOS.
- Merge por humano, sempre.

6. Implementação

Abrir a porta e confirmar que abriu

São duas coisas diferentes, e o app não junta as duas. Ele só diz que a porta abriu depois de ler o estado e ver que concorda com o comando. Se a fechadura não responde, o app diz que não sabe, em vez de inventar.

Nesta versão os comandos que mexem na porta vêm desligados. A porta é de uma conta compartilhada, e errar para o lado cauteloso é a escolha certa.

O que barra o merge

Testes	314
ADRs	27
Cobertura	61,5 %

O que impede um merge é o teste de arquitetura, a suíte de testes e a checagem das telas. Cobertura é medida e publicada, mas não barra.

Como usei IA

Para escrever código; as decisões continuaram sendo minhas. Nada entra sem teste e nada é mesclado sozinho. Toda decisão importante virou um documento curto com o motivo, e onde a IA errou está anotado, junto com o que eu fiz para corrigir.

Escrever rápido é fácil. O trabalho é revisar o que foi escrito.

7. Próximos passos

No fluxo do app

- **Início do vídeo mais robusto:** reconexão melhor, indicação de progresso real, e cair para uma alternativa antes de falhar.
- **Ferramentas de câmera:** captura de imagem, gravação de trecho, tela cheia e controles de qualidade.
- **Favoritar dispositivos**, para o que se usa todo dia ficar no topo da lista.
- **Grupos e ambientes** (sala, garagem, entrada), para navegar por lugar em vez de por lista.
- **Atalhos** para abrir a câmera ou a fechadura direto da tela inicial do celular.

No projeto

- **Mais regras de arquitetura.** Das onze que desenhei, cinco estão implementadas. A que mais falta impede uma feature de importar a outra, e é justamente ela que protege o caminho para módulos por feature descrito na seção 4.
- **Capturas de tela no próprio pull request.** Hoje elas são geradas e ficam num arquivo para baixar; deveriam aparecer na página do PR, para revisar mudança de tela olhando em vez de imaginando.
- **Comparação automática dessas capturas**, para uma tela quebrada reprovar o PR sozinha.